

# SistecreditoTEF

Calidad, seguridad y arquitectura del módulo TEF que conecta HioPosCloud con Credinet, para los almacenes Permoda en Colombia.

## STACK

**.NET 10 · MAUI · Android 13+**

## INTEGRACIÓN

**HioPosCloud ↔ CREDINET**

## TIPO

**Módulo TEF · Crédito**

## RELEASE

**Julio 2026**

7

#### ENDPOINTS CREDINET

REST + JSON

11

#### INTENT-FILTERS

HioPosCloud actions

14

#### EXTRAS ECO-BACK

Por cada TRANSACTION

30

#### TESTS AUTOMATIZADOS

xUnit + CapturingHandler

1

##### Contrato es ley

El contrato con el POS es ley. Cada extra del Intent de salida está validado.

2

##### Claves secretas

Las claves nunca se imprimen en logs. Las cédulas se enmascaran siempre.

3

##### Falla con causa

Si algo se rompe, se rompe con causa clara. `ApiResult<T>` sin excepciones.

Figura 1 — Los 3 principios rectores que guían cada decisión técnica del proyecto.

# Lo que vas a encontrar

Un recorrido por las decisiones de arquitectura, seguridad, estándares y pruebas que sostienen este módulo.

|                                                                                                                                                                                     |                                                                                                                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>1</div> <div><b>Seguridad del software</b><br/>4 capas · datos sensibles · threat model</div> | <div>2</div> <div><b>Principios de diseño</b><br/>SOLID · DRY · KISS con ejemplos</div>     |
| <div>3</div> <div><b>Arquitectura: el porqué</b><br/>Capas · flujo de transacción</div>            | <div>4</div> <div><b>Estándares de codificación</b><br/>Naming · records · async · DI</div> |
| <div>5</div> <div><b>Pruebas realizadas</b><br/>Inventario · cobertura · capacidad</div>           | <div>6</div> <div><b>Conclusiones</b><br/>Qué está listo · qué falta</div>                  |

## Seguridad del software

La seguridad se modela en **cuatro capas concéntricas de defensa**: si una capa falla, las demás siguen protegiendo. Este modelo viene del manual de HioPosCloud §6.1 y se implementa capa por capa con medidas concretas en código.



## Datos sensibles: política de tratamiento

Aplicamos la matriz del manual §6.2 con tres políticas claras: **prohibido**, **enmascarado** y **acceptable**.

| DATO                                  | RIESGO | POLÍTICA            | IMPLEMENTACIÓN                                                              |
|---------------------------------------|--------|---------------------|-----------------------------------------------------------------------------|
| <code>Subscription-Key</code>         | ALTA   | Nunca en logs       | <code>HttpLoggingHandler.IsSensitiveHeader()</code> → <b>***REDACTED***</b> |
| Cédula<br>( <code>idDocument</code> ) | MEDIA  | Enmascarada en URL  | Regex enmascara → <b>*****1234</b>                                          |
| OTP / Token                           | ALTA   | Nunca persistido    | Solo en <code>OtpViewModel.CodigoOtp</code> (memoria)                       |
| <code>creditId</code>                 | MEDIA  | OK en logs internos | Reconciliación + auditoría Permoda                                          |
| Mobile / Email                        | MEDIA  | Solo memoria        | En <code>Client</code> in-memory; <code>state.Clear()</code> tras TX        |

## Medidas implementadas

Cada medida está respaldada por una clase específica del proyecto que la implementa.

**Certificate Pinning**

`CertificatePinning.cs` valida el SPKI SHA-256 del certificado contra `cfg.CertificatePins`.  
Vacío = TLS estándar; con pins = seguro.

**SQLCipher**

`SQLitePCLRaw.bundle_e_sqlcipher` activo. La BD `idempotency.db3` queda cifrada AES-256 en reposo.

**Redacción Subscription-Key**

`IsSensitiveHeader()` oculta `Ocp-Apim-Subscription-Key` y `Authorization` en headers, body y logs.



### Enmascarar cédula

Regex `(idDocument=)(\d+)`  
enmascara los primeros dígitos  
dejando solo los últimos 4.



### Reintentos con Polly

`CredinetHttpPolicies.cs` reintent  
solo fallos transitorios y solo en GETs  
idempotentes. Excluye OTP y POST.



### state.Clear() tras TX

Antes de  
`FinishWithResult` se  
limpia cliente, crédito,  
documento y monto de la  
memoria.

## Threat model — ¿qué ataques cubrimos?

**Mentalidad:**  
Cada fila es un escenario realista, no teórico. El atacante está dentro o cerca del POS, o tiene acceso a la red WiFi de la tienda.

| AMENAZA                                  | VECTOR                                     | MITIGACIÓN                                                         |
|------------------------------------------|--------------------------------------------|--------------------------------------------------------------------|
| App falsa suplanta identidad             | Otro APK declara los mismos intent-filters | ICG valida el SHA-256 del firmante en CloudLicense                 |
| Robo de Subscription-Key en tránsito     | MITM en LAN del POS                        | TLS 1.2+ + Certificate Pinning SPKI                                |
| Fuga de key por logs                     | Logcat/dumpsys revela el header            | <code>HttpLoggingHandler</code> redacta antes de persistir         |
| Lectura de BD local                      | POS robado o USB debugging                 | SQLCipher AES-256: ilegible sin passphrase                         |
| Doble cobro (re-ataque)                  | POS re-dispara TRANSACTION tras timeout    | <code>SqliteIdempotencyStore</code> indexa por <code>SaleId</code> |
| XML malicioso en SellerData/DocumentPath | Manipulación del XML por terceros          | Extracción defensiva + <code>Escape()</code> en ReceiptBuilder     |

## 3 DESIGN PRINCIPLES Principios de diseño

El proyecto se adhiere a tres familias de principios: **SOLID** (diseño orientado a objetos), **DRY** (una sola fuente de verdad) y **KISS** (simplicidad sobre generalización prematura).

### Los 5 principios SOLID



Figura 3 — Los 5 principios SOLID. Cada color representa un pilar de diseño distinto.

#### S · Single

##### Single Responsibility

Cada clase tiene una sola razón para cambiar. `SistecreditoService` orquesta lógica de dominio; no sabe nada de HTTP, SQLite ni Android Intents.

```
public class SistecreditoService {  
    // Solo orquesta, no sabe de Android  
    private readonly ICredinetRepository _repo;  
    private readonly IAuditLogger _audit;  
    private readonly IIdempotencyStore _idempotency;  
}
```

#### O · Open

##### Open / Closed

Extensiones sin modificar. Los enums `DocumentType` exponen mapeos como métodos de extensión.

```
public enum DocumentType { CedulaCiudadania, CedulaExtranjeria }  
  
public static class DocumentTypeExtensions {  
    public static string Code(this DocumentType doc)  
    {  
        return doc switch  
        {  
            DocumentType.CedulaCiudadania => "C",  
            DocumentType.CedulaExtranjeria => "E",  
        };  
    }  
}
```

#### L · Liskov

##### Liskov Substitution

Las implementaciones son intercambiables. `ITransactionResultHandler` tiene una implementación real y una `NoOp` para tests.

```
public interface ITransactionResultHandler {  
    void FinishWithResult(HioposResponse response);  
}  
  
// Producción  
public class AndroidTransactionResultHandler : ITransactionResultHandler { ... }  
  
// Tests  
internal class NoOpTransactionResultHandler : ITransactionResultHandler { }
```

#### I · Interface

##### Interface Segregation

Interfaces pequeñas y específicas. Separamos `ITransactionStateStore` e `INavigationService`.

```
// El VM pide solo lo que necesita  
public partial class ConfirmacionViewModel {  
    ITransactionStateStore state,  
    ITransactionResultHandler resultHandler,  
    ReceiptBuilder receiptBuilder,  
    INavigationService nav) : Observable {  
    }
```

#### D · Dependency Inversion

##### Dependency Inversion

Las dependencias apuntan a abstracciones, no a concreciones. Toda la inyección se configura en `MauiProgram.CreateMauiApp`.

```
// MauiProgram.cs - un solo lugar, todas las dependencias  
builder.Services.AddSingleton<ICredinetRepository, CredinetRepository>();  
builder.Services.AddSingleton<IAuditLogger, BroadcastAuditLogger>();  
builder.Services.AddSingleton<ITransactionResultHandler, AndroidTransactionResultHandler>();  
// Tests inyectan fakes sin recompilar la app.
```

DRY

### Don't Repeat Yourself

Cada constante vive en un único lugar. Cambiar el nombre del APK se hace en **una sola línea**:

```
// HioposActions.cs
public const string ApkName = "sistecredito";
private const string Prefix = "icg.actions.electronicpayment." + ApkName + ".";

public const string Initialize = Prefix + "INITIALIZE";
public const string Transaction = Prefix + "TRANSACTION";
// ... las 11 actions se generan desde aquí
```

Otras "single sources of truth": `ApiConfig` (URL/headers), `HioposCapabilities` (18 flags), `CredinetMapper`, `ReceiptBuilder`.

KISS

### Keep It Simple, Stupid

Preferimos la solución obvia. Tipos de retorno seguros de flujo.

```
// En vez de lanzar y atrapar excepciones
public abstract record ApiResult<T> {
    public sealed record Ok<U>(U Data)
    public sealed record Failure<U>(ApiException U)
}

// El VM hace pattern matching, sin try/catch
switch (result) {
    case ApiResult<Client>.Ok<Client> ok when ok.Data != null:
        // ...
    case ApiResult<Client>.Failure<Client> failure:
        // ...
}
```

## Arquitectura: el porqué

La elección de separar *View* ↔ *ViewModel* ↔ *UseCase* ↔ *Service* ↔ *Repository* no fue caprichosa: cada límite existe para resolver un problema concreto.

### Las 5 capas de la aplicación

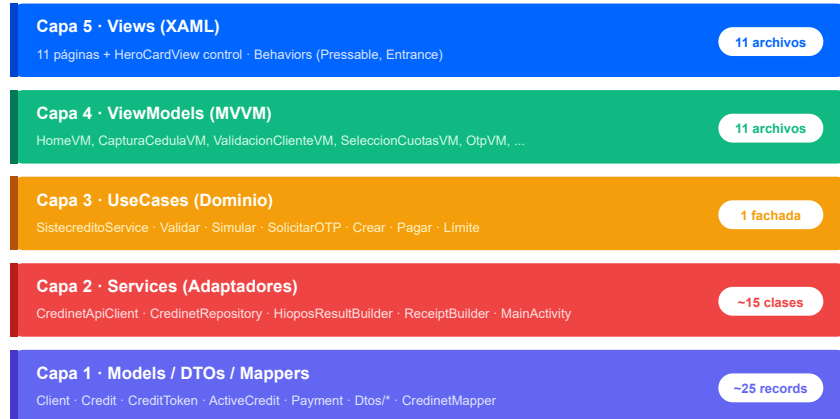


Figura 4 — Las 5 capas de la aplicación. Cada capa depende solo de la capa inferior.

#### Testabilidad.

El `SistecreditoService` se testea con `FakeRepo` que no toca red. Tests corren offline en milisegundos.

#### Mantenibilidad.

Cambiar la UI no toca la lógica de dominio. Cambiar la API no toca la UI.

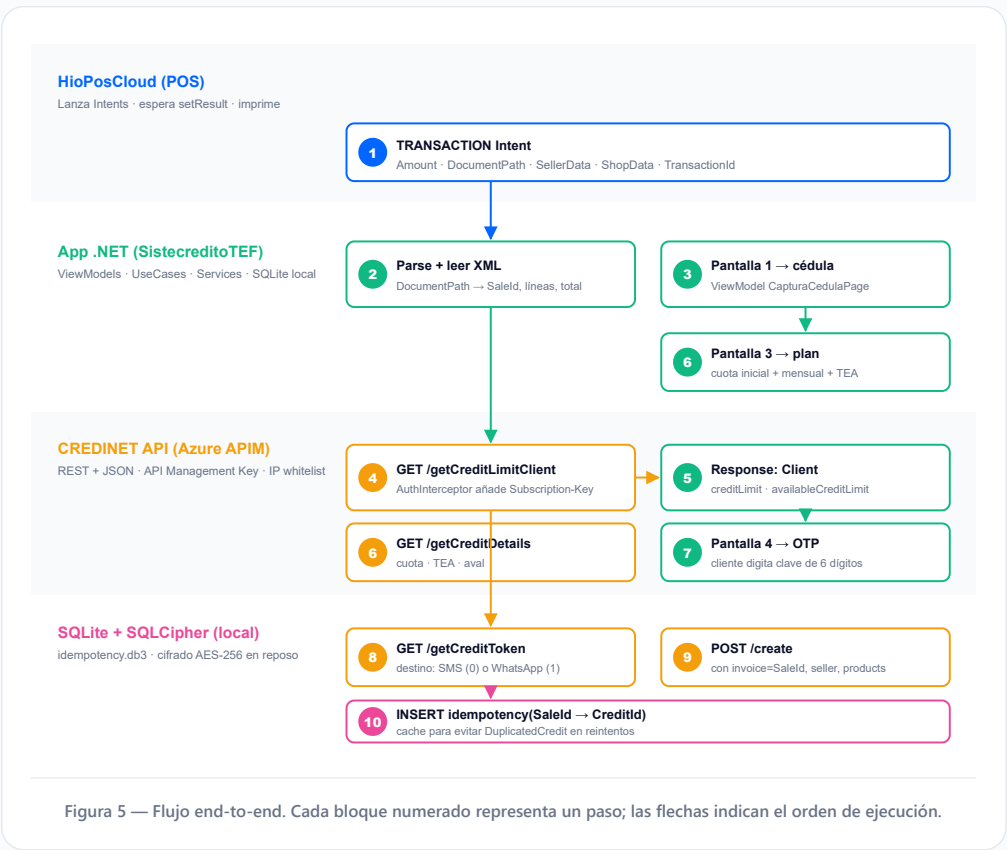
#### Sustituibilidad.

Reemplazar `HttpLoggingHandler` por Serilog no rompe nada — solo se cambia el binding.

#### Demo vs Producción.

`SistecreditoService` detecta `IsSyntheticDemo` y devuelve datos sintéticos sin red. La UI es la misma.

# Flujo de una transacción completa



5

CODING STANDARDS

# Estándares de codificación

Convención revisada en cada PR. Lo que no está acá, se resuelve por mayoría de lo existente.

## Convenciones de nombres

| ELEMENTO             | CONVENCIÓN                | EJEMPLO DEL PROYECTO                                                                               |
|----------------------|---------------------------|----------------------------------------------------------------------------------------------------|
| Clases / records     | PascalCase, singular      | <code>SistecreditoService</code> , <code>CreditTokenDto</code> , <code>ApiResponse&lt;T&gt;</code> |
| Interfaces           | IPascalCase               | <code>ICredinetApi</code> , <code>IAuditLogger</code> , <code>ITransactionStateStore</code>        |
| Métodos públicos     | PascalCase, verbos        | <code>ValidarClienteAsync</code> , <code>BuildBehavior</code>                                      |
| Propiedades          | PascalCase                | <code>CreditLimit</code> , <code>AvailableCreditLimit</code>                                       |
| Campos privados      | <code>_camelCase</code>   | <code>_repo</code> , <code>_audit</code> , <code>_config</code> , <code>_pendingIntent</code>      |
| Parámetros / locales | camelCase                 | <code>creditValue</code> , <code>idDocument</code> , <code>tipoDocumento</code>                    |
| Constantes           | PascalCase                | <code>ApkName</code> , <code>SandboxSubscriptionKey</code>                                         |
| Enums                | PascalCase                | <code>enum DocumentType { CedulaCiudadania, ... }</code>                                           |
| Async                | Sufijo <code>Async</code> | <code>CrearCreditoAsync</code> , <code>GetCreditLimitClientAsync</code>                            |

### Records, inmutabilidad y DTOs

Usamos `record` para todos los modelos de dominio y DTOs. Por qué: **inmutabilidad**, **igualdad por valor**, **serialización trivial**, **sintaxis concisa**.

```
public record Credit(  
    string TypeDocument,  
    string IdDocument,  
    string CreditId,  
    int CreditNumber,  
    double EffectiveAnnualRate,  
    // ...  
);
```

### Async/await y cancelación

Reglas: todo I/O es async, sufijo `Async` obligatorio, sin `ConfigureAwait(false)` en UI.

```
// VM: continuacion en UI thread para bindings + Shell  
public async Task ValidarAsync() {  
    var result = await service.ValidarClienteAsync(  
        TipoDocumento, NumeroDocumento);  
    switch (result) {  
        case ApiResponse<Client>.Ok<Client> ok:  
            // bindings actualizados, Shell.GoToAsync  
    }  
}
```

### Nullable reference types

`<Nullable>enable</Nullable>` activo. DTOs nullable (`string?` , `double?` ); dominio no-null con defaults aplicados en mapper.

```
// DTO: nullable porque el backend puede omitir  
public record ClientDto(  
    [JsonPropertyName("creditLimit")] double? CreditLimit,  
    [JsonPropertyName("email")] string? Email);  
  
// Dominio: no-null (mapper aplica defaults)  
public record Client(  
    DocumentType DocumentType,  
    string DocumentId, // nunca null  
    double CreditLimit; // nunca null
```

### Inyección de dependencias

Toda dependencia se inyecta por constructor. Nunca Service Locator. El `MauiProgram` .

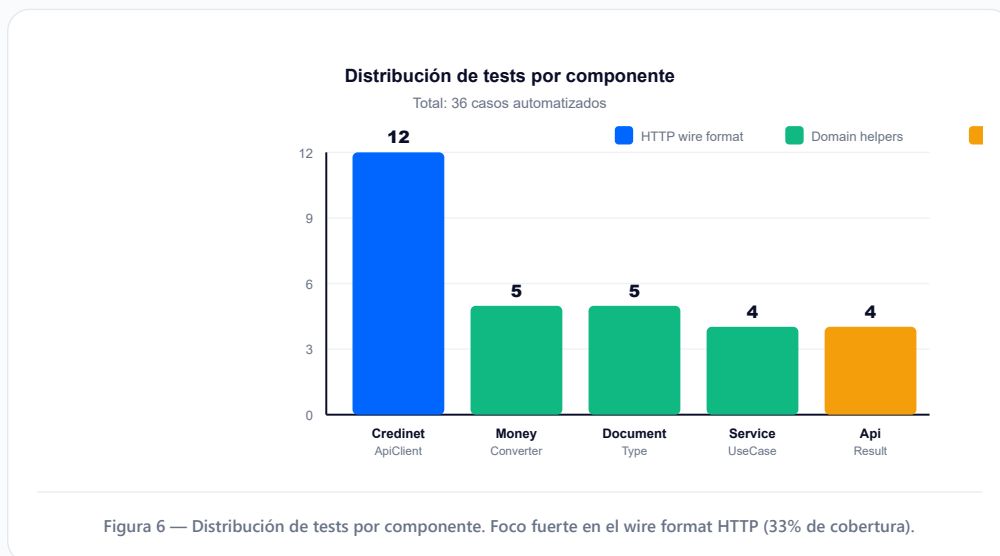
```
// Singletons: estado compartido, HTTP, SQLite  
builder.Services.AddSingleton<ICredinetRepository, CredinetR  
builder.Services.AddSingleton<ITransactionStateStore, Transa  
  
// Transients: VMs y Pages (uno por navegación)  
builder.Services.AddTransient<HomeViewModel>();  
builder.Services.AddTransient<CapturaCedulaPage>();
```

## Inventario de tests automatizados

Framework: **xUnit**. Handlers: **CapturingHandler** (mock que captura el `HttpRequestMessage` real antes de salir por red).  
Fakes: implementaciones a mano de las interfaces (sin Moq, consistente con la versión Kotlin original).

| CAPA                | TEST                                            | VERIFICA                                                                                    |
|---------------------|-------------------------------------------------|---------------------------------------------------------------------------------------------|
| ApiResult<T>        | <code>Ok_carries_data</code>                    | El record <code>Ok&lt;U&gt;</code> preserva el dato                                         |
|                     | <code>Map_on_Ok / Failure</code>                | Map genérico transforma el dato, propaga Failure                                            |
| MoneyConverter      | <code>FromCentsToPesos</code> ×5                | Casos límite: 50000000, 5000, 1, null, "abc"                                                |
| DocumentType        | <code>FromCode / Code</code> ×5                 | CC ↔ CedulaCiudadania, CE ↔ CedulaExtranjeria                                               |
| CredinetMapper      | <code>ClientDto_toDomain</code> ×3              | DTO lleno, DTO nulls → defaults seguros, CE                                                 |
| HTTP                | <code>AuthInterceptorTests</code> ×3            | Header <code>Ocp-Apim-Subscription-Key</code> , <code>Accept</code> , no SCLocation/country |
|                     | <code>CredinetApiClientRequestTests</code> ×12  | URL exacta, método, query string, body JSON, omisión de storeId/destination                 |
| SistecreditoService | <code>ValidarCliente_audit_log_OK / FAIL</code> | Emite <code>SC_VALIDATE_OK</code> / <code>SC_VALIDATE_FAIL</code> con código 221            |
|                     | <code>CrearCredito_idempotencia</code>          | Hit de caché por SaleId, persistencia tras crear                                            |

## Cobertura por componente

 ¿Cuánto tarda una transacción típica?

Medimos cada paso del flujo para que sepas exactamente dónde se va el tiempo. La respuesta corta: **la parte automática toma ~1.3 segundos**. El resto depende del cajero.

LO QUE HACE LA APP

1.3<sub>s</sub>

≈ un parpadeo largo · más rápido que un estornudo



LO QUE TARDA EL CAJERO

5–15<sub>s</sub>

≈ leer un WhatsApp + digitar 6 dígitos





## Herramientas

```
# Opción A: k6 (script JS, contra CREDINET directo)
k6 run --vus 10 --duration 60s load-test.js

# Opción B: NBomber en .NET (más cómodo si seguimos el stack)
dotnet run --project tools/LoadTest/LoadTest.csproj -c Release

# Opción C: smoke real en terminal HioPos
# 1. Lanzar app desde el launcher de la terminal
# 2. Cédula: 1026260942 → Validar
# 3. Monto: 500.000 · 12 cuotas → Calcular
# 4. Solicitar OTP (cualquier destino)
# 5. OTP: 123456 (sandbox)
# 6. Cronometrar con cronómetro del celular ⌚
```

## Conclusiones

SistecreditoTEF es un módulo pequeño en líneas de código (~3.500 LoC + tests) pero denso en responsabilidades contractuales: 11 acciones HioPos, 7 endpoints CREDINET, eco de 14 extras, persistencia cifrada, auditoría broadcast.

Las decisiones de diseño — capas MVVM + service, contratos tipados con `ApiResponse<T>`, inyección estricta, configuración externalizada, security-by-default — no son accidentes: son la respuesta directa a los problemas que el manual de HioPos y Credinet describen.



### Lo que ya está resuelto

- Funcionalidad end-to-end completa
- 7 endpoints CREDINET
- 11 intent-filters HioPos
- SQLCipher activo en BD local
- Cert Pinning + Polly + Redacción
- Idempotencia por SaleId
- Eco de 14/14 extras a HioPos
- Demo mode + Producción por config



### Pendientes para go-live

- Pruebas de carga formales (k6/NBomber)
- Migrar Preferences → SecureStorage (opcional)
- Tablas `audit_log` + `error_log` en SQLite
- Serilog opcional (logs estructurados)
- Hash de cédula en logs (no solo URL)
- Pin SPKI real de `api.credinet.co`
- Coordinación con ICG (CloudLicense)
- Coordinación con Sistecredito (key prod + IP whitelist)

# SistecreditoTEF

Documento técnico de Calidad y Seguridad del Software - v1.0.0

**Stack** .NET 10 MAUI · Android 13+    **Integración** HioPosCloud ↔ CREDINET    **Generado** Julio 2026    **Cliente** Permoda · Colombia